

LAPORAN PRATIKUM PEMROGRAMAN BERBASIS OBJEK

MODUL 2: Relasi Antar Objek (Class Diagram & Asosiasi)

Pekan 2

DI SUSUN OLEH :

ABDUL KARIM ALGAZALI

NIM 2511532029

DOSEN PENGAMPU : NURFIAH, S.ST, M.KOM

ASISTEN LABORATORIUM : Dawi Syauqi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa, karena atas rahmat dan izin-Nya laporan praktikum “Modul 2: Relasi Antar Objek (Class Diagram & Asosiasi)” ini dapat diselesaikan dengan baik. Saya mengucapkan terima kasih sebesar-besarnya kepada Bapak Nurfiah, S.ST, M.Kom selaku dosen pengampu mata kuliah Pemrograman Berorientasi Objek serta kepada asisten laboratorium yang telah membimbing pelaksanaan praktikum ini.

Laporan ini disusun sebagai bentuk dokumentasi kegiatan praktikum Pekan 2 yang bertujuan untuk memahami relasi antar objek dalam pemrograman berorientasi objek, khususnya relasi asosiasi one-to-many antara objek Rekening dan objek Transaksi. Pada praktikum ini, sistem perbankan mini yang telah dibuat pada pekan sebelumnya dikembangkan dengan menambahkan fitur pencatatan riwayat transaksi serta fitur cetak mutasi rekening.

Melalui praktikum ini, saya belajar bagaimana membaca dan menerjemahkan class diagram ke dalam kode program Java, membuat relasi antar kelas, serta mengelola koleksi objek di dalam objek lain menggunakan struktur data dinamis ArrayList.

Saya menyadari bahwa penulisan laporan praktikum ini masih jauh dari kata sempurna. Oleh karena itu, saya dengan rendah hati menerima saran, masukan, dan kritik guna penyempurnaan laporan ini.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang	3
1.2 Tujuan	4
1.3 Manfaat	4
BAB II PEMBAHASAN.....	5
2.1 Analisis Class diagram dan Relasi	5
2.2 Implementasi Class Transaksi.....	5
2.3 Implementasi Class Rekening.....	6
2.4 Implementasi Class Main.....	9
2.5 Mekanisme Program Keja.....	13
BAB III TUGAS.....	14
3.1 Integrasi Rekam Jejak Penarikan	14
3.2 Fitur Cetak Mutasi Rekening.....	15
BAB IV KESIMPULAN.....	16
3.1 Kesimpulan	16
3.2 Saran	16
DAFTAR PUSTAKA.....	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam Pada praktikum pekan pertama, telah dibangun sistem perbankan mini sederhana dengan menggunakan konsep dasar Pemrograman Berorientasi Objek, seperti class, object, constructor, attribute, dan method. Sistem tersebut sudah mampu melakukan beberapa operasi dasar, seperti membuka rekening baru, setor tunai, tarik tunai, melihat informasi rekening, dan mengganti akun. Namun, sistem tersebut belum memiliki kemampuan untuk menyimpan riwayat transaksi yang terjadi pada setiap rekening.

Dalam aplikasi perbankan yang lebih nyata, setiap aktivitas keuangan seperti setor tunai dan tarik tunai sebaiknya dicatat sebagai sebuah transaksi. Catatan transaksi ini berguna untuk keperluan audit, pengecekan riwayat, dan peningkatan pengalaman pengguna. Oleh karena itu, pada praktikum pekan kedua ini dikembangkan sistem yang mampu merekam setiap transaksi yang berhasil dilakukan oleh pengguna.

Untuk mewujudkan hal tersebut, digunakan konsep relasi antar objek. Dalam Pemrograman Berorientasi Objek, sebuah objek dapat memiliki relasi dengan objek lain. Pada kasus ini, objek Rekening memiliki relasi dengan objek Transaksi. Satu rekening dapat memiliki banyak transaksi, sehingga relasi ini merupakan relasi asosiasi one-to-many. Relasi ini diimplementasikan menggunakan bantuan struktur data dinamis ArrayList yang disimpan di dalam objek Rekening.

Selain itu, praktikum ini juga menekankan kemampuan untuk membaca dan menerjemahkan class diagram ke dalam kode program. Class diagram digunakan sebagai representasi visual struktur kelas, atribut, method, serta relasi antar kelas sebelum diimplementasikan ke dalam bahasa pemrograman Java.

1.2 Tujuan

1. Mampu membaca dan menerjemahkan class diagram ke dalam kode program Java.
2. Memahami dan mampu mengimplementasikan relasi asosiasi one-to-many antar objek.
3. Mampu mengelola koleksi objek di dalam objek lain menggunakan tipe data bentukan dan struktur data dinamis ArrayList.
4. Mampu menambahkan fitur pencatatan riwayat transaksi pada sistem perbankan.
5. Mampu membuat fitur cetak mutasi rekening berdasarkan riwayat transaksi yang telah tersimpan.

1.3 Manfaat

1. Meningkatkan pemahaman mengenai relasi antar objek dalam pemrograman berorientasi objek.
2. Memberikan kemampuan untuk mengimplementasikan hubungan one-to-many menggunakan ArrayList.
3. Melatih mahasiswa dalam mengembangkan sistem yang lebih kompleks dari sistem sebelumnya.
4. Meningkatkan pemahaman mengenai pentingnya penyimpanan riwayat transaksi dalam sebuah sistem perbankan.
5. Memperkuat kemampuan dalam membaca class diagram dan mengubahnya menjadi implementasi kode program.

BAB II

PEMBAHASAN

2.1 Analisis class diagram dan relasi asosiasi

Pada praktikum pekan ke-2 ini, sistem perbankan mini dikembangkan dengan menambahkan satu class baru, yaitu class Transaksi. Class ini digunakan untuk merepresentasikan satu aktivitas transaksi yang terjadi pada sebuah rekening. Class Transaksi memiliki beberapa atribut dasar, yaitu idTransaksi, jenis, dan nominal.

Relasi antara class Rekening dan class Transaksi merupakan relasi asosiasi one-to-many. Artinya, satu objek Rekening dapat memiliki banyak objek Transaksi. Relasi ini menggambarkan hubungan has-a, yaitu sebuah rekening memiliki riwayat transaksi.

2.2 Implementasi Class Transaksi

Class Transaksi merupakan class baru yang ditambahkan pada praktikum pekan ke-2. Class ini berfungsi untuk menyimpan data satu transaksi yang berhasil dilakukan oleh pengguna. Setiap transaksi memiliki identitas unik, jenis transaksi, dan nominal transaksi. Jenis transaksi dibedakan menjadi dua, yaitu:

1. Kredit untuk transaksi setor tunai.
2. Debit untuk transaksi tarik tunai.

Code

<pre>package pekan2; public class Transaksi { String idTransaksi; String jenis; double nominal; public Transaksi(String id, String jenis, double nominal) { this.idTransaksi = id; this.jenis = jenis; this.nominal = nominal; } public void cetakTransaksi() { System.out.println("ID Transaksi: " + idTransaksi + ", Jenis: " + jenis + ", Nominal: Rp " + nominal); } public void cetakDetail() {</pre>

```

        cetakTransaksi();
    }
}

```

Class Transaksi memiliki tiga atribut, yaitu:

1. idTransaksi bertipe String, digunakan untuk menyimpan kode unik transaksi.
2. jenis bertipe String, digunakan untuk menyimpan jenis transaksi, yaitu Kredit atau Debit.
3. nominal bertipe double, digunakan untuk menyimpan jumlah uang yang ditransaksikan.

Constructor pada class Transaksi menerima tiga parameter, yaitu id, jenis, dan nominal. Constructor ini akan dipanggil setiap kali objek transaksi baru dibuat, baik saat setor tunai maupun tarik tunai.

Method cetakDetail() digunakan untuk menampilkan detail satu transaksi ke layar.

Method ini nantinya dipanggil oleh method cetakMutasi() yang berada di dalam class Rekening.

2.3 Implementasi Class Rekening

Class Rekening merupakan pengembangan dari class yang telah dibuat pada praktikum pekan pertama. Pada pekan ini, class Rekening ditambahkan atribut baru berupa daftar riwayat transaksi. Selain itu, method setorTunai() dan tarikTunai() juga diperbaiki agar setiap transaksi yang berhasil dicatat ke dalam riwayat transaksi.

Berikut adalah implementasi class Rekening:

Code
<pre> package pekan2; import java.util.ArrayList; public class Rekening { String nomorRekening; String namaPemilik; double saldo; ArrayList<Transaksi> riwayatTransaksi; public Rekening(String nomor,String nama, double saldoAwal) { nomorRekening = nomor; namaPemilik = nama; saldo = saldoAwal; this.riwayatTransaksi = new ArrayList<>(); System.out.println("Rekening atas nama" + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo); </pre>

```

    }

    public String getNomorRekening() {
        return nomorRekening;
    }

    public void setorTunai(double nominal) {
        if ( nominal > 0) {
            saldo += nominal;
            String idTrx = "TRX-S" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);
            System.out.println("Saldo tunai Rp" + nominal + " berhasil, Saldo saat ini: Rp"
+ saldo);
        } else {
            System.out.println("Gagal : nominal setor harus lebih dari 0");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Gagal : nominal tarik harus minimal 10000");
        } else if (saldo - nominal < 10000) {
            System.out.println("Gagal: Saldo tidak mencukupi (saldo minimal tersisa
Rp10.000).");
        } else {
            saldo -= nominal;
            String idTrx = "TRX-T" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
            riwayatTransaksi.add(trxBaru);
            System.out.println("Tarik tunai Rp" + nominal + " berhasil, Saldo saat ini: Rp"
+ saldo);
        }
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Terakhir: Rp" + saldo);
        System.out.println("-----");
    }

    public void cetakMutasi() {
        if (riwayatTransaksi.isEmpty()) {
            System.out.println("Belum ada transaksi pada rekening ini");
        } else {
            for (Transaksi transaksi : riwayatTransaksi) {
                transaksi.cetakDetail();
            }
        }
    }

```

```
}  
}  
}
```

Beberapa perubahan penting pada class Rekening adalah sebagai berikut:

1. Penambahan atribut riwayat transaksi

Atribut berikut ditambahkan ke dalam class Rekening:

```
ArrayList<Transaksi> riwayatTransaksi;
```

Atribut ini digunakan untuk menyimpan seluruh objek transaksi yang terjadi pada rekening tersebut.

2. Inisialisasi ArrayList di dalam constructor

Pada constructor, dilakukan inisialisasi objek ArrayList seperti berikut:

```
this.riwayatTransaksi = new ArrayList<>();
```

Inisialisasi ini penting karena jika tidak dilakukan, daftar riwayat transaksi akan bernilai null, sehingga program dapat mengalami NullPointerException saat menambahkan transaksi pertama.

3. Perekaman transaksi pada method setorTunai

Ketika setor tunai berhasil, program akan membuat objek Transaksi baru dengan jenis "Kredit" dan menambakkannya ke dalam riwayatTransaksi.

```
String idTrx = "TRX-S-" + System.currentTimeMillis();  
Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);  
riwayatTransaksi.add(trxBaru);
```

4. Perekaman transaksi pada method tarikTunai

Ketika tarik tunai berhasil, program akan membuat objek Transaksi baru dengan jenis "Debit" dan menambakkannya ke dalam riwayatTransaksi.

```
String idTrx = "TRX-T-" + System.currentTimeMillis();  
Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);  
riwayatTransaksi.add(trxBaru);
```

5. Penambahan method cetakMutasi

Method cetakMutasi() digunakan untuk menampilkan seluruh transaksi yang tersimpan di dalam rekening. Jika belum ada transaksi, program akan menampilkan pesan bahwa belum ada transaksi pada rekening tersebut.

2.4 Implementasi Class Main

Class Main merupakan class utama yang digunakan untuk menjalankan program melalui command line interface. Pada class ini, program menampilkan menu utama, menerima input pengguna, dan mengeksekusi operasi yang sesuai dengan menu yang dipilih.

Pada pekan ke-2 ini, ditambahkan satu menu baru, yaitu menu 6. Cetak Mutasi (Riwayat). Menu ini digunakan untuk menampilkan riwayat transaksi dari rekening yang sedang aktif.

Berikut adalah implementasi class Main:

Code:

```
package pekan2;

import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor tunai");
            System.out.println("3. Tarik tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("6. Cetak Mutasi (Riwayat)");
            System.out.println("0. Keluar");
            System.out.print("Pilih Menu : ");

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print("Masukkan nomor rekening: ");
                    String no = input.nextLine();

                    boolean sudahTerdaftar = false;
```

```

        for (Rekening r : daftarRekening) {
            if (r.getNomorRekening().equals(no)) {
                sudahTerdaftar = true;
                break;
            }
        }

        if (sudahTerdaftar) {
            System.out.println("Error: Nomor rekening sudah terdaftar!");
        } else {
            System.out.print("Masukkan nama pemilik: ");
            String nama = input.nextLine();
            System.out.print("Masukkan saldo awal: ");
            double saldo = input.nextDouble();

            Rekening rekeningBaru = new Rekening(no, nama, saldo);
            daftarRekening.add(rekeningBaru);
            akunAktif = rekeningBaru;
            System.out.println("Rekening berhasil dibuat dan langsung
aktif!");
        }
        break;

        case 2:
            if (akunAktif == null) {
                System.out.println("Error: Mohon maaf, anda belum memiliki
nomor rekening / belum login!");
            } else {
                System.out.print("Masukkan nominal setor: ");
                double setor = input.nextDouble();
                akunAktif.setorTunai(setor);
            }
            break;

        case 3:
            if (akunAktif == null) {
                System.out.println("Error: Mohon maaf, anda belum memiliki
nomor rekening / belum login!");
            } else {
                System.out.print("Masukkan nominal penarikan: ");
                double tarik = input.nextDouble();
                akunAktif.tarikTunai(tarik);
            }
            break;

        case 4:
            if (akunAktif == null) {
                System.out.println("Error: Anda belum membuka rekening /
belum login!");
            }

```

```

        } else {
            akunAktif.cekInformasi();
        }
        break;

    case 5:
        if (daftarRekening.isEmpty()) {
            System.out.println("Belum ada rekening yang terdaftar. Silakan
buka rekening baru terlebih dahulu.");
        } else {
            System.out.print("Masukkan nomor rekening yang ingin
diaktifkan: ");

            String noTujuan = input.nextLine();
            boolean ditemukan = false;

            for (Rekening r : daftarRekening) {
                if (r.getNomorRekening().equals(noTujuan)) {
                    akunAktif = r;
                    ditemukan = true;
                    System.out.println("Berhasil ganti akun!");
                    break;
                }
            }

            if (!ditemukan) {
                System.out.println("Nomor rekening tidak ditemukan!");
            }
        }
        break;

    case 6:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum membuka rekening /
belum login");
        } else {
            akunAktif.cetakMutasi();
        }
        break;

    case 0:
        isRunning = false;
        System.out.println("Sistem ditutup. Terima kasih!");
        break;

    default:
        System.out.println("Pilihan tidak valid");
    }
}

```

```
        input.close();
    }
}
```

Pada class Main, terdapat beberapa variabel penting, yaitu:

1. daftarRekening

Merupakan ArrayList<Rekening> yang digunakan untuk menyimpan seluruh rekening yang telah dibuat oleh pengguna.

2. akunAktif

Merupakan referensi ke objek rekening yang sedang digunakan oleh pengguna saat ini.

3. isRunning

Merupakan variabel boolean yang digunakan untuk mengontrol perulangan menu utama. Program akan terus menampilkan menu utama selama isRunning bernilai true. Ketika pengguna memilih menu keluar, yaitu menu 0, maka nilai isRunning diubah menjadi false, sehingga perulangan berhenti dan program selesai.

Penambahan menu ke-6 pada class Main memungkinkan pengguna untuk melihat riwayat transaksi dari rekening yang sedang aktif.

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening / belum login");
    } else {
        akunAktif.cetakMutasi();
    }
    break;
```

Jika pengguna belum membuka atau memilih rekening, sistem akan menampilkan pesan error. Namun, jika sudah ada rekening aktif, sistem akan memanggil method cetakMutasi() dari objek rekening tersebut.

2.5 Mekanisme Kerja Program

Secara umum, mekanisme kerja program pada praktikum pekan ke-2 ini adalah sebagai berikut:

1. Program menampilkan menu utama sistem perbankan mini.
2. Pengguna memilih menu yang tersedia.
3. Jika pengguna memilih menu buka rekening baru, program akan meminta input nomor rekening, nama pemilik, dan saldo awal.
4. Program memeriksa apakah nomor rekening sudah terdaftar atau belum.
5. Jika nomor rekening belum terdaftar, program membuat objek Rekening baru dan menyimpannya ke dalam daftarRekening.
6. Rekening yang baru dibuat langsung dijadikan rekening aktif.
7. Jika pengguna melakukan setor tunai, program akan memperbarui saldo dan membuat objek Transaksi dengan jenis Kredit.
8. Jika pengguna melakukan tarik tunai dan validasi berhasil, program akan memperbarui saldo dan membuat objek Transaksi dengan jenis Debit.
9. Setiap transaksi yang berhasil akan disimpan ke dalam riwayatTransaksi milik rekening aktif.
10. Jika pengguna memilih menu cetak mutasi, program akan menampilkan seluruh riwayat transaksi yang tersimpan di dalam rekening aktif.

Dengan mekanisme ini, setiap rekening memiliki riwayat transaksinya masing-masing, sehingga data transaksi antar rekening tidak saling tercampur.

BAB III

TUGAS

3.1 Integrasi Rekam Jejak Penarikan

Tugas pertama pada praktikum pekan ke-2 adalah mengintegrasikan perekaman riwayat transaksi ke dalam method tarikTunai(double nominal) pada class Rekening.

Method tarikTunai() sudah ada sejak praktikum pekan pertama. Namun, pada pekan ini method tersebut diperbaiki agar setiap penarikan yang berhasil dicatat sebagai objek transaksi.

Terdapat dua validasi utama pada method tarikTunai(), yaitu:

1. Nominal penarikan minimal Rp10.000.
2. Saldo setelah penarikan tidak boleh kurang dari Rp10.000.

Jika kedua validasi tersebut terpenuhi, maka transaksi penarikan dianggap berhasil. Pada saat transaksi berhasil, program membuat objek Transaksi baru dengan jenis "Debit" dan kode transaksi yang menggunakan prefiks "TRX-T".

Berikut adalah implementasi method tarikTunai() yang telah dilengkapi dengan pencatatan riwayat transaksi:

```
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Gagal: nominal tarik harus minimal 10000");
    } else if (saldo - nominal < 10000) {
        System.out.println("Gagal: Saldo tidak mencukupi " +
            "(saldo minimal tersisa Rp10.000).");
    } else {
        saldo -= nominal;

        String idTrx = "TRX-T" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Tarik tunai Rp" + nominal +
            " berhasil, Saldo saat ini: Rp" + saldo);
    }
}
```

Pada method tersebut, objek transaksi hanya dibuat jika penarikan berhasil. Dengan demikian, transaksi yang gagal tidak akan dicatat ke dalam riwayat transaksi. Hal ini sesuai dengan aturan pada modul praktikum yang menyatakan bahwa transaksi hanya boleh dicatat jika penarikan berhasil dan saldo mencukupi.

3.2 Implementasi Fitur Multi Akun

Tugas kedua pada praktikum pekan ke-2 adalah membuat fitur cetak mutasi rekening. Fitur ini digunakan untuk menampilkan seluruh riwayat transaksi yang pernah terjadi pada sebuah rekening.

Untuk mengimplementasikan fitur ini, ditambahkan method baru pada class Rekening, yaitu method cetakMutasi(). Method ini menggunakan struktur perulangan for-each untuk menelusuri seluruh objek transaksi yang tersimpan di dalam riwayatTransaksi.

Jika riwayat transaksi masih kosong, program akan menampilkan pesan bahwa belum ada transaksi pada rekening tersebut. Namun, jika sudah ada transaksi, program akan menampilkan detail setiap transaksi dengan memanggil method cetakDetail() dari masing-masing objek transaksi.

Berikut adalah implementasi method cetakMutasi():

```
public void cetakMutasi() {  
    if (riwayatTransaksi.isEmpty()) {  
        System.out.println("Belum ada transaksi pada rekening ini");  
    } else {  
        for (Transaksi transaksi : riwayatTransaksi) {  
            transaksi.cetakDetail();  
        }  
    }  
}
```

Method ini kemudian dipanggil pada class Main melalui menu ke-6, yaitu menu Cetak Mutasi (Riwayat).

Berikut adalah implementasi menu cetak mutasi pada class Main:

```
case 6:  
    if (akunAktif == null) {  
        System.out.println("Error: Anda belum membuka rekening / belum login");  
    } else {  
        akunAktif.cetakMutasi();  
    }  
    break;
```

Sebelum method cetakMutasi() dipanggil, program melakukan pengecekan terlebih dahulu terhadap objek akunAktif. Jika akunAktif masih bernilai null, artinya pengguna belum membuka rekening atau belum memilih rekening yang aktif. Dalam kondisi tersebut, program menampilkan pesan error.

Jika pengguna sudah memiliki rekening aktif, maka method cetakMutasi() akan dieksekusi dan seluruh riwayat transaksi pada rekening tersebut akan ditampilkan. Dengan adanya fitur ini, pengguna dapat melihat seluruh aktivitas setor tunai dan tarik tunai yang telah dilakukan pada rekening yang sedang aktif.

BAB IV KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa class diagram berperan penting sebagai gambaran awal struktur class, atribut, method, serta relasi antar class sebelum diimplementasikan ke dalam kode program Java. Pada praktikum pekan ke-2 ini, relasi antar objek diterapkan melalui hubungan asosiasi one-to-many antara class Rekening dan class Transaksi, di mana satu objek rekening dapat memiliki banyak objek transaksi. Relasi tersebut diimplementasikan dengan menambahkan atribut berupa koleksi `ArrayList<Transaksi>` di dalam class Rekening. Inisialisasi `ArrayList` di dalam constructor juga diperlukan agar objek riwayat transaksi siap digunakan dan tidak menyebabkan error `NullPointerException` saat transaksi pertama kali ditambahkan.

Selain itu, praktikum ini menunjukkan bahwa setiap transaksi yang berhasil, baik setor tunai maupun tarik tunai, perlu dicatat sebagai objek transaksi untuk menjaga riwayat aktivitas rekening tetap terstruktur. Transaksi setor tunai dicatat dengan jenis Kredit, sedangkan transaksi tarik tunai dicatat dengan jenis Debit dan hanya direkam apabila penarikan berhasil memenuhi validasi saldo. Fitur cetak mutasi yang diimplementasikan menggunakan perulangan `for-each` memungkinkan sistem menampilkan seluruh riwayat transaksi milik rekening yang sedang aktif. Dengan adanya relasi antar objek ini, sistem perbankan mini menjadi lebih terstruktur, fleksibel, dan lebih mendekati kebutuhan aplikasi perbankan nyata.

3.2 Saran

1. Penggunaan visualisasi class diagram dan struktur relasi antar objek akan sangat membantu mahasiswa dalam memahami hubungan asosiasi one-to-many antara objek Rekening dan objek Transaksi, serta bagaimana `ArrayList<Transaksi>` disimpan di dalam setiap objek rekening. Visualisasi ini juga dapat memperjelas konsep relasi has-a, alur pembuatan objek transaksi, dan penyebab terjadinya `NullPointerException` apabila koleksi objek tidak diinisialisasi terlebih dahulu di dalam constructor.
2. Penambahan materi pra-praktikum melalui iLearn mengenai class diagram, relasi asosiasi, struktur data Collection khususnya `ArrayList`, serta pengelolaan exception seperti `NullPointerException` dan `InputMismatchException` tetap perlu dilanjutkan agar mahasiswa memiliki fondasi konseptual yang matang sebelum mengimplementasikan sistem yang melibatkan banyak objek dan riwayat transaksi. Materi ini dapat disertai contoh kasus sederhana agar mahasiswa lebih siap menghadapi tantangan saat praktikum.

DAFTAR PUSTAKA

- [1] Nurfiah. Modul Praktikum PBO Java: Praktikum 2 Relasi Antar Objek (Class Diagram & Asosiasi). [Diakses: 2026].